ChatGPT 5.2

I'd like to discuss a key topic for my project. I use fastEmbed all-MiniLM-L6-v2, as it is a lightweight model. Now for keyword/tag extraction functionality, I'm gearing towards using TF-IDF, but I know that a limitation of this model is that non-significant keywords can be extracted. And I was wondering whether there would be other solutions, maybe even compatible with my current infrastructure.

TF-IDF is a solid baseline for tags, but you're right about the main failure mode: it happily returns *rare* tokens that are not actually *important* (IDs, boilerplate, formatting artifacts, etc.). If you want higher-precision tags while staying compatible with a lightweight, local-first pipeline (and your existing MiniLM embeddings/Qdrant setup), these options tend to work well.

1) Keep TF-IDF, but fix its precision problems

These improvements usually give the biggest ROI without adding heavy dependencies.

- **Candidate filtering (most important)**
  - restrict to **noun phrases / n-grams** (1–3) instead of raw tokens
  - remove numbers/IDs, very short tokens, and "shape" noise (`abc123` , `v1.2.3` , hex, etc.)
  - add a **domain stoplist** (e.g., "section", "table", "figure", "document", "shall", "hereby" in governance docs)
- **Sublinear TF + better normalization**
  - `use sublinear_tf=True + norm='l2'`
  - prefer **char n-grams** only when you have lots of typos/variants; otherwise they increase junk.
- **Use BM25 instead of TF-IDF**
  - BM25 is often a direct upgrade for keywording; it reduces some TF-IDF pathologies and is still lightweight.
- **"TF-IDF within parent"**
  - Since you already have parent+chunks: compute tags at the *parent* level, but use chunk statistics for candidate discovery and stability (e.g., candidates must appear in ≥k chunks, or across multiple sections).

2) Embedding-assisted re-ranking (very compatible with your current stack)

Use TF-IDF (or BM25) only to generate *candidates*, then re-rank by semantic similarity using the same MiniLM embeddings you already produce.

Workflow

1. Extract ~200 candidates (phrases) with TF-IDF/BM25.
2. Embed:
   - the parent text (or summary of it)
   - each candidate phrase
3. Score candidates by:
   - cosine(parent_embedding, phrase_embedding)
   - optionally penalize very generic phrases (global frequency)

This usually knocks out "rare but irrelevant" tokens because they don't semantically align with the document.

Why it works here

- No new model.
- Uses existing fastEmbed/Qdrant-compatible vectors.
- Deterministic, fast, and tunable.

3) KeyBERT-style tagging (but local and cheap)

KeyBERT is basically the idea above packaged: **candidate extraction + embedding similarity**. You don't need the library if you want to keep dependencies minimal; the algorithm is simple.

Enhancements that matter:

- **Maximal Marginal Relevance (MMR)**: increases diversity so you don't get 10 near-synonyms.
- **Use noun-phrase candidates** (spaCy/Stanza) instead of raw n-grams if you can afford it.

4) Graph-based keywording: RAKE / TextRank / PositionRank

These are lightweight and surprisingly effective in some corpora.

- **RAKE**: good for multiword phrases; depends heavily on stopwords quality.
- **TextRank / PositionRank**: graph centrality; tends to surface "concept" phrases better than TF-IDF, but can still pick boilerplate unless filtered.
- Works well as either:
  - an alternative candidate generator, or
  - a second signal combined with TF-IDF + embedding re-rank.

5) Supervised-lite: train a tiny tagger from your own data

If you can label even a few hundred docs with good tags, you can train something small:

- linear model on TF-IDF features, or
- shallow classifier on MiniLM embeddings.

This yields much higher precision and is still lightweight. You can also do "weak supervision":

- accept tags that match a controlled vocabulary / taxonomy
- accept phrases that occur in headings, bold text, or defined terms sections (common in governance docs)

6) Controlled vocabulary + synonym mapping (high precision, low recall)

For governance/criteria documents, a hybrid often wins:

- maintain a **taxonomy** of allowed tags (risk type, control family, model stage, validation artifacts, etc.)
- detect tags via:
  - exact match + synonyms
  - embedding similarity to taxonomy labels (fast, robust)

This avoids nonsense tags entirely and is very compatible with your infrastructure.

Recommended setup for your project (practical hybrid)

Given you already chunk, embed, and store in Qdrant:

1. **Candidate extraction**: BM25 or TF-IDF on noun phrases / 1–3 grams with strong filters.
2. **Semantic re-rank**: MiniLM cosine similarity against parent embedding (or section embeddings).
3. **Diversity**: MMR to keep tags distinct.
4. **Optional**: taxonomy "snap" (map tag → canonical label via nearest taxonomy embedding).

This stays lightweight, local, and uses the embedding model you already have.

If you tell me 3 specifics, I can pin down the best configuration

- typical document types (policies, checklists, code, mixed?)
- desired tags per document (e.g., 5–15?) and whether multiword phrases are preferred
- whether you already have (or can build) a small controlled vocabulary / taxonomy

1) Sure: we store in qdrant using 3 collections projects, libraries, scratchbook.
- projects contains code and related files (i.e. markdown), but other documents such as pdf or ePub belonging to the project are stored in the libraries collection (these are typically academic papers or background information that are better fitted in a library than diluting values in the project itself.
- libraries is basically a knowledge base, it contains articles, books, and other reference information (example could be: numerical methods, design patterns, etc), it could cover a large gamut of knowledge and not be limited to a few specific topics
- scratchbook is use to take notes of ideas, of research, of draft PRD, basically to store information that is not yet ready to be transformed in actual implementation.

All collections are multi-tenant. Projects tenant's are projects, the libraries are documents, and the scratchbook are either global (no project yet) or per project.

2) I'm thinking about extracting keywords and tags, the way I see it (but not a strong position) is to get a small number of tags for grouping, and a larger list of keywords for searches. The grouping aspect would be for instance to be able to group similar projects and thus be able to do searches in the current project and to expand to similar projects grouped via a relatively small set of tags. Keywords being a larger collection that would allow to tighten searches. I am not sure what to think about multi-word phrases for either category, however I could see a value in using "quasi-summaries" to be embedded with chunks and parents.

3) I am looking into unsupervised operations, without curation. Though this would not prevent the collection of keywords into one or multiple baskets (the basket being defined by the best tag).

4) If possible, I'd be interested into building hierarchies, for instance "Science - Physics - Classical Physics - Classical Electrodynamics", again unsupervised

5) Ideally I'd like to keep using FastEmbed since it works well with all-MiniLM-L6-v2, but if another model can be used for keywording I don't mind having FastEmbed with more than one model, as long as they remain small.

Target outcomes (mapped to your constraints)

- **Tags (small set, for grouping / expansion across similar projects or documents)**
- **Keywords (larger set, for tightening searches)**
- **Unsupervised**, no manual curation
- **Hierarchies** like `Science → Physics → …`
- **Lightweight**, compatible with FastEmbed + all-MiniLM-L6-v2 and Qdrant

Below is a pipeline that stays “small-model + unsupervised” and leverages what you already store (parent + chunks).

---

1) Use a two-stage approach: candidate extraction → embedding rerank

This is the simplest way to avoid TF-IDF's "rare but useless" tokens without introducing LLM keywording.

1A) Candidate extraction (fast, lexical)

Do this per **parent document** (or per section) to generate 100–300 candidates.

Use one of:

- **BM25** (often better than TF-IDF out of the box)
- **TF-IDF** with strong filters
- **YAKE** or **RAKE** as alternative candidate generators (they're lightweight and good at phrases)

Critical filters (these matter more than the scorer):

- favor **noun phrases** (best), or at least **1–3 word n-grams**
- drop tokens that match patterns like IDs/hashes/versions/paths (`abc123` , `0x…` , `v1.2.3` , `/usr/bin` , etc.)
- stoplist boilerplate ("introduction", "section", "references", "shall", "hereby", etc.)
- keep a per-tenant "junk lexicon" automatically grown from the top global frequent candidates

1B) Embedding rerank (semantic, using your existing model)

Embed:

- the **parent** (or its "quasi-summary", see below)
- each candidate phrase

Score candidates by cosine similarity to the parent embedding, and keep:

- **Top K keywords** (e.g., 30–80)
- **Top T tags** (e.g., 5–12) after diversity selection (MMR)

This single step eliminates most non-significant TF-IDF picks because they don't semantically align.

---

2) Make "quasi-summaries" without a generative model

You can get summary-like vectors/text for each parent without LLMs:

Option A (recommended): embed-only summary vector

- Parent vector = mean (or weighted mean) of **chunk vectors**
- Weight chunks by:
  - lexical salience (BM25/TF-IDF score of chunk vs document)
  - or "centrality" (chunk similarity to other chunks)

You can store:

- `parent_summary_vector` (for grouping / tags)
- optionally a short extracted text "gist":

Option B: extractive "gist text"

- pick the 2–5 most central/salient chunks (or sentences) and concatenate as a "gist"
- embed that gist for tagging
- store gist text as a human-readable field

This is cheap and often works better than embedding the entire parent text when parents are large.

---

3) Separate "keywords" vs "tags" by intent, not by algorithm

Use the same candidate pool, then split by two different selection rules.

Keywords (bigger list; search tightening)

Keep candidates with:

- high semantic score to parent
- acceptable document-frequency (not too generic per-tenant)
- allow more redundancy

Typical: 30–100 per parent.

Tags (small set; grouping & expansion)

Take the same scored candidates, but add:

- **diversity (MMR)** so tags cover different aspects
- optionally require multi-occurrence across chunks (stability)

Typical: 5–12 per parent.

---

4) "Baskets" (assign keywords under the best tag) — unsupervised

Once you have tag embeddings and keyword embeddings:

- For each keyword `k`, assign it to tag `t` with max cosine similarity.
- If the best similarity is below a threshold, put it in "misc".

This gives you "baskets defined by the best tag" with no curation.

---

5) Unsupervised hierarchies (Science → Physics → …) without a taxonomy

You can build hierarchies from **tag embeddings** with standard clustering.

Practical approach: hierarchical clustering on tag vectors

Per tenant (or per collection+tenant), maintain a set of **canonical tag nodes**.

1. Collect tag candidates across documents (e.g., all tags from last N docs).
2. Deduplicate by embedding similarity (merge near-duplicates into a canonical node).
3. Run **agglomerative clustering** on canonical tag vectors.
4. Cut the dendrogram at multiple levels to form:
   - Level 1: broad topics
   - Level 2: subtopics
   - Level 3: fine-grained

Store each node with:

- centroid vector
- label (see below)
- children IDs

How to label nodes without an LLM

Use **extractive labeling**:

- For each cluster, pick the top phrases closest to centroid, then choose:
  - shortest meaningful phrase, or
  - most frequent among cluster members, or
  - a 2–3 phrase "label set"

This yields usable hierarchies (not perfect human taxonomy, but functional).

Important note

Without curation, you should expect:

- some odd cluster names
- occasional "mixed" nodes at higher levels

  This is still very useful for navigation and expansion queries.

---

6) Where this fits into your 3 collections

You likely want different behavior by collection:

projects

- emphasize code identifiers less; prefer concepts (modules, patterns, algorithms)
- candidate extraction should treat code carefully (identifiers can become junk)
- use AST-aware chunking (which you already do)

libraries

- best results with noun-phrase candidates + semantic rerank
- hierarchies are especially valuable here (broad knowledge base)

scratchbook

- tags drift; use higher diversity and lower stability thresholds
- consider time-decay when building canonical tag nodes (recent ideas matter more)

All of this is multi-tenant safe if you build:

- per-tenant IDF/statistics
- per-tenant canonical tag graph

---

7) Models: keep MiniLM, optionally add one more small model

You can keep FastEmbed + all-MiniLM-L6-v2 for everything.

If you want an additional small model, the only compelling reason is:

- better phrase embeddings or multilingual coverage

But in practice, your biggest gains will come from:

- noun-phrase candidate quality
- semantic rerank
- dedupe + clustering

So I would not add a second model until after you've measured precision/utility.

Minimal concrete spec (what to implement first)

1. **Candidate generator**: BM25 or TF-IDF with filters (prefer noun phrases if possible).
2. **Semantic rerank**: embed parent-summary + candidates with MiniLM; cosine score.
3. **Split outputs**:
   - keywords = top 50 by score (with basic DF penalty)
   - tags = top 8 with MMR + stability across chunks
4. **Keyword baskets**: assign keyword → nearest tag.
5. **Hierarchy v1**: nightly job:
   - canonicalize tags (merge near-duplicates)
   - hierarchical clustering
   - label nodes extractively
   - store as a per-tenant "topic tree" in Qdrant (or separate metadata store)

If you want, I can also outline the exact metadata schema to store:

- per-doc tags/keywords/baskets
- canonical tag nodes
- hierarchy edges
- per-tenant DF stats

  so you can query "current project → similar tags → expand search to similar projects" efficiently in

Qdrant.

When it comes to code files, should they be excluded or should they be treated under a different umbrella? i.e. the language name?

Short answer: **do not exclude code**, but **treat it under a different umbrella with a different extraction strategy**. Code carries strong structural signal, but naïve TF-IDF will mostly extract identifiers and noise.

You should separate:

- **Concept tags** (algorithms, patterns, domains)
- **Structural/code tags** (language, framework, architecture)
- **Identifier keywords** (class/function names, symbols)

These behave differently.

---

1. Code should not be excluded

In your `projects` collection especially, code is often the *most semantically meaningful artifact*. It defines:

- architecture
- algorithms
- dependencies
- patterns
- domain vocabulary

But it must not be processed like prose.

---

2. Treat code under a structured pipeline

Instead of raw TF-IDF over source files, extract structured signals first.

2.1 Always add language as a top-level tag

Yes — language should be a **first-class structural tag**, not discovered by TF-IDF.

Example tags:

- `language:rust`
- `language:python`
- `language:lua`
- `language:swift`

Do not treat language as a keyword. It's a grouping dimension.

2.2 Extract structured candidates from AST (preferred)

For code files, candidate generation should be AST-aware:

Extract:

- class names
- struct names
- enum names

- trait / interface names
- function names (public only ideally)
- module names
- annotations / decorators
- imports

These become **identifier keywords**, not semantic tags.

Then:

- normalize (snake_case → space separated words)
- split camelCase
- remove trivial suffixes (`Impl` , `Manager` , `Helper` , `Utils`)

Example:

```
PrimeSieveMultiOverlap
→ ["prime sieve", "multi overlap"]
```

Then feed those phrases into the same embedding rerank step.

---

3. Separate 3 types of tags for code

A) Structural tags (low cardinality)

Automatically derived:

- language
- framework (detected from imports)
- architecture patterns (if detectable)
- test/benchmark/tooling

Example:

- `language:rust`
- `framework:tokio`
- `pattern:repository`
- `layer:data-access`

These are excellent for grouping projects.

B) Concept tags (semantic)

These should be extracted using:

1. AST-aware phrase extraction
2. Plus docstrings/comments
3. Plus semantic rerank against parent summary

Examples:

- `prime generation`
- `vector indexing`
- `grpc protocol`
- `hierarchical clustering`

These are cross-project grouping tags.

C) Identifier keywords (high volume)

These support tightening searches:

- function names
- type names
- internal terminology

These should:

- not pollute tags
- not be used for hierarchy building
- be stored for keyword search only

---

4. How this fits your three collections

projects

Code is primary.

Pipeline:

- AST extraction
- docstring + comment lexical extraction
- semantic rerank
- separate structural vs conceptual tags

libraries

Mostly prose.
Use noun-phrase extraction + semantic rerank.

scratchbook

Free-form.
Treat as prose, but use higher diversity threshold.

---

5. Should code tags include the language umbrella?

Yes — but as metadata, not discovered tags.

Better model:

Instead of:

ini Copy code

```ini
tags = ["rust", "vector indexing", "grpc"]
```

Use:

makefile Copy code

```makefile
structural_tags = {
  language: "rust",
  frameworks: ["tokio", "serde"]
}

concept_tags = ["vector indexing", "grpc protocol"]
```

Language is too coarse to behave like a semantic tag; it's a filter dimension.

6. Should code participate in hierarchy building?

Yes — but only with concept tags.

Do not build hierarchy from:

- identifiers
- language names
- internal module names

Build hierarchy only from:

- deduplicated concept tags
- optionally documentation-derived phrases

Otherwise you'll get nonsense clusters like:

```rust
serde → derive → json → impl
```

7. Important: code needs a different stoplist

You should automatically suppress:

- `impl`
- `mod`
- `struct`
- `pub`
- `self`
- `async`
- `let`
- trivial variable names (`data` , `value` , `item` , `res`)

This dramatically improves quality.

8. Advanced but powerful option

If you want higher quality concept extraction from code without a big model:

Build a **symbol co-occurrence graph**:

- nodes = extracted phrases
- edges = appear in same file/module

Rank by:

- graph centrality
- plus semantic similarity to parent summary

This surfaces real architectural concepts better than TF-IDF.

9. Clear recommendation for your system

Do NOT exclude code.

Instead:

For code files:

1. Detect language → structural tag.
2. Extract AST identifiers → candidate pool.
3. Extract comment/docstring noun phrases.
4. Normalize identifiers to phrases.
5. Embed and semantic rerank.
6. Split:
   - concept_tags (small set)
   - identifier_keywords (larger set)

For prose:

Use noun phrases + semantic rerank.

For hierarchy:

Use concept_tags only.

If you want, I can next:

- design a clean metadata schema for Qdrant that separates structural, concept, and identifier layers, or

I forgot to mention that the code is already pre-processed with treesitter (to define non-arbitrary chunks) and metadata is enriched with lsp data when available. How would this fit into your suggestion?

That fits very well; it lets you replace "token/phrase heuristics" with "symbol- and scope-aware candidates," which is exactly what makes code keywording/tagging work.

1) Use Tree-sitter chunks as the unit of evidence

Instead of extracting candidates from the whole file, do it per chunk type:

- `function`, `method`, `class/struct`, `module`, `test`, `comment/docstring`, `import` blocks
- Aggregate upward: **chunk → file → project parent**

This gives you stability signals ("appears in many chunks") and prevents one big file from dominating.

2) Use LSP as the primary source of candidates (when available)

Treat LSP symbols as **high-quality candidate keywords**:

Candidate pools:

- **API surface**: exported/public symbols, types, traits/interfaces
- **Core concepts**: frequently referenced symbols (by reference count)
- **Dependencies**: imports / resolved modules / packages

Normalize for phrase-like use:

- split camelCase/snake_case
- de-prefix namespaces
- drop trivial suffixes (`Impl`, `Manager`, `Util`) via a configurable rule set

Then classify:

- **Identifier keywords**: symbol names (kept mostly as-is + normalized variant)
- **Concept candidates**: symbol-derived phrases + doc/comment noun phrases

3) Add “semantic re-rank” using your existing embeddings, but change what you embed

For code, embedding the raw source is noisy. Prefer embedding:

- **chunk summaries**: docstring + signature + a short extracted “symbol context”
- or **symbol context strings** built from LSP:
  - definition name + kind + containing module
  - top referenced types it interacts with
  - doc comment (if any)

Example context string to embed (no LLM needed):

vbnet Copy code

```vbnet
Symbol: PrimeSieve
Kind: struct
Module: primes::sieve
Doc: Generates primes using segmented sieve with overlap windows
Refs: find_n_primes_multi, WindowOverlap, HashSet
```

Compute:

- `chunk_vector` (from context string)
- `file_vector` (mean/weighted mean of chunk vectors)
- `project_vector` (mean/weighted mean of file vectors)

Then re-rank candidate tags/keywords by cosine similarity to the appropriate vector:

- chunk-level keywords: similarity to `chunk_vector`
- file/project tags: similarity to `file_vector` or `project_vector`

4) Derive structural tags from LSP + metadata, not from extraction

These become your “umbrella” dimensions:

- `language:rust`
- `build:cargo` / `build:npm`
- `framework:tokio`, `framework:react` (from dependency graph/imports)
- `layer:test` / `layer:benchmark` (from path + chunk types + test frameworks)

These are excellent for grouping and should not compete with semantic tags.

5) Make “concept tags” from cross-symbol patterns, not raw identifiers

With LSP you can do better than TF-IDF:

Signals for concept tags:

- **high fan-in types**: types referenced across many files/modules
- **high fan-out modules**: modules importing many others (architectural hubs)
- **co-occurrence**: symbols that frequently appear together across chunks

Workflow:

1. Start with symbol candidates (normalized phrases).
2. Filter by stability:
   - appears in ≥ N chunks, or
   - referenced in ≥ N locations, or
   - spans ≥ N files
3. Semantic rerank against `project_vector`.
4. Apply diversity (MMR) for final tag set.

This yields tags like “grpc”, “vector index”, “sieve”, “hierarchical clustering” rather than “prime_sieve_impl”.

6) Keywords vs tags using your existing evidence hierarchy

- **Keywords** (bigger list): top candidates per chunk/file with weaker stability constraints.
- **Tags** (small list): only candidates that are stable across chunks/files + high semantic score + diverse.

Practical defaults:

- keywords: 30–100 per file (or 10–30 per chunk if you store them)
- tags: 5–12 per project, 3–8 per file

7) Hierarchies become feasible because you have better nodes

Build hierarchies from **canonical concept tags** (not identifiers):

- Create canonical tag nodes per tenant by merging near-duplicates via embedding similarity.
- Cluster canonical nodes hierarchically (agglomerative).
- Label clusters extractively (pick 1–3 closest canonical phrases).

LSP helps because your concept tags will be less noisy, so the tree is cleaner.

8) Where TF-IDF/BM25 still helps in code

Use it only on:

- **docstrings/comments/README/markdown**
- (optionally) symbol-context strings

Let LSP/tree-sitter drive the candidate set; let lexical methods add missing natural-language phrases.

Net change to the earlier recommendation

- Candidate extraction for code should be **LSP-first**, Tree-sitter-scoped.
- Embedding rerank stays, but you embed **symbol context**, not raw code.
- Language/framework become **structural tags** from metadata, not discovered.

If you want the next step, I can specify:

- an exact candidate scoring formula combining (references, chunk coverage, file coverage, semantic similarity),
- and a compact Qdrant payload schema to store per-chunk keywords + per-file tags + per-project tags without inflating payload size.

Lastly, while we use all-MiniLM-L6-v2 for dense embedding, is BM25 the best way to generate sparse vectors or is there another way?

Thought for 7s

BM25 is the best **classical** way to produce sparse vectors, but it is not the best overall if you can afford a small neural sparse model. In your stack (Qdrant + FastEmbed + all-MiniLM-L6-v2 dense), the usual upgrade path is:

Options for sparse vectors (from simplest to strongest)

1) BM25 sparse vectors (baseline; strong keyword precision)

- Pros: very fast, robust, interpretable; great for exact/rare terms.
- Cons: no semantic expansion; synonyms/morphology only via stemming/config.
- Good default if you want minimal moving parts.

Qdrant supports BM25 sparse vectors natively and via FastEmbed. qdrant.tech +1

2) SPLADE / SPLADE++ sparse embeddings (often better recall than BM25)

- These are **learned sparse** vectors: still sparse/interpretable, but they do **contextual lexical expansion** (adds related terms with weights).
- Pros: tends to beat BM25 on many retrieval tasks (especially when synonyms matter).
- Cons: heavier than BM25 (but still "small model" territory).

FastEmbed supports SPLADE++ models (e.g., `prithivida/Splade_PP_en_v1`) and Qdrant documents SPLADE for sparse vector generation. qdrant.tech +2

3) "BM42" / lightweight sparse models aligned with MiniLM (very compatible)

FastEmbed lists a lightweight sparse model `Qdrant/bm42-all-minilm-l6-v2-attentions` , designed as a small sparse embedder (and still requires IDF statistics). qdrant.github.io
If you want "BM25-like simplicity" but slightly more semantic behavior, these lightweight sparse models are typically the next thing to try (benchmark on your corpus).

What I would recommend for your setup

If you want the safest first step

Use BM25 sparse + your MiniLM dense in hybrid retrieval, and combine with RRF or weighted fusion. Qdrant's hybrid search guidance uses this approach. qdrant.tech +1

If you want better recall without a big model

Try **SPLADE++ sparse vectors** (FastEmbed-supported) + MiniLM dense, same fusion strategy. This usually improves "keyword but not exact wording" queries. qdrant.tech +2

If you want to stay "MiniLM-family" and small

Try **BM42** (`Qdrant/bm42-all-minilm-l6-v2-attentions`) as the sparse side, then compare against BM25 and SPLADE++ on your evaluation set. qdrant.github.io

---

Practical notes that matter in multi-tenant Qdrant

- **BM25-style sparse models require IDF statistics**; Qdrant/FastEmbed flows account for this, but you should treat IDF as **tenant-scoped** (or at least collection+tenant-scoped) to avoid cross-tenant skew. (FastEmbed marks BM25/BM42 as `requires_idf` .) qdrant.github.io
- If you ingest continuously, plan how/when IDF updates occur (Qdrant notes BM25 conversion can be handled in Qdrant; implementation details vary by client/version). qdrant.tech +1

Decision rule

- Choose **BM25** if: exact term match is the main pain point; you want minimal compute/complexity.
- Choose **SPLADE++** if: users often type related terms/synonyms; you want better recall with still-sparse vectors.
- Choose **BM42** if: you want a small sparse model that pairs naturally with MiniLM and stays lightweight.

If you describe your typical queries (short keywords vs sentence queries; code-symbol queries vs prose), I can give a concrete pick (BM25 vs SPLADE++ vs BM42) and a fusion weighting scheme for tags/keywords vs dense semantics.

Lastly, an adjacent question: while I'm working on Intel Mac, I am also

targeting Apple Silicon. Should I consider opting for a specific implementation and model version to benefit the most from Apple's performance?

Thought for 24s

Yes—if you want to benefit from Apple Silicon (GPU/ANE), you should plan for **two execution paths**: a portable default (what you have now) and an Apple-optimized backend. The *model* can stay "MiniLM-class"; the difference is mostly **runtime + format**.

What matters most on Apple Silicon

1) Avoid Rosetta; ship/execute native arm64

This is table-stakes. If your Python env or ONNX runtime is running under x86_64 emulation, you'll lose most benefits.

2) FastEmbed today is primarily ONNX Runtime CPU on macOS

FastEmbed is built around ONNX Runtime inference. qdrant.github.io
As of mid-2025, FastEmbed maintainers were still discussing lack of a dedicated Mac/Silicon build / GPU acceleration, and there are open requests about using M-series GPUs. GitHub +1
So: **expect CPU-heavy embedding inference** if you stay purely in FastEmbed on macOS.

3) To use Apple's accelerators, you typically need Core ML / MLX

- **ONNX Runtime CoreML Execution Provider** exists and is recommended for Apple devices with ANE for performance, but operator coverage / conversion behavior can vary by model. ONNX Runtime +1
- **MLX** is Apple Silicon–oriented and integrates with Transformers; it keeps arrays in shared memory and is designed to run efficiently on Apple Silicon. Hugging Face

Recommended strategy for your project

Baseline (cross-platform, stable)

- Keep **FastEmbed + all-MiniLM-L6-v2** for dense embeddings (ONNX Runtime path).
- Keep sparse as **BM25/BM42/SPLADE++** as you choose (sparse computation won't materially benefit from GPU/ANE anyway; it's mostly token/statistics work).

Apple-optimized optional backend (for Apple Silicon targets)

Add a pluggable "embedding engine" with two implementations:

A) ONNX Runtime + CoreML EP (best if it works for MiniLM)

- Convert/export your MiniLM encoder to ONNX (if not already), run with providers:
  - `CoreMLExecutionProvider` then `CPUExecutionProvider` fallback.
- Benefit: you keep the same ONNX ecosystem; easier to integrate than fully switching stacks. ONNX Runtime +1
- Risk: CoreML EP may fall back to CPU for unsupported ops, or change precision (needs benchmarking).

B) Core ML model (mlmodel) for embeddings

- Convert the encoder to Core ML (coremltools pipeline) and run via Core ML directly.
- Benefit: most direct route to ANE/GPU.
- Cost: you maintain a second model artifact and conversion pipeline.

C) MLX-based inference (if you're already using HF models elsewhere)

- Benefit: very Apple Silicon–friendly runtime and tooling. Hugging Face
- Cost: you may need to adapt your embedding model format/implementation to MLX.

Model/version choices for Apple performance

- Prefer **small encoders** (MiniLM size is fine); the main gains come from runtime/accelerator use, not a special "Apple" version of MiniLM.
- If you add a second model, do it for **quality vs speed**, not "Apple-specific." Keep dimensions and normalization consistent so your Qdrant vectors remain comparable within a collection.

Practical guidance

- Implement an **embedding microservice/daemon** (you already have gRPC): runtime is selected at startup:
  - Intel Mac / Linux: FastEmbed ONNX CPU
  - Apple Silicon: try CoreML EP → fallback to ONNX CPU
- Benchmark on Apple Silicon using your *actual* workload (chunk sizes, batch sizes). The winner is often decided by batching and tokenization overhead more than raw matmul speed.

One adjacent note

Qdrant itself can be built with GPU indexing using Vulkan, and docs explicitly mention Apple Silicon compatibility for Vulkan 1.3 devices—this is about indexing/search acceleration, not embedding inference. qdrant.tech

If you tell me your language/runtime for the embedding daemon (Python vs Rust vs Swift) and whether you're willing to ship a CoreML-converted artifact, I can recommend the most realistic Apple path.

I'm using rust and we are building an all-inclusive binary for the daemon, i.e. ONNX being statically linked in the binary (and recompiled for Intel Mac as no bin is available). Qdrant is installed separately and is used as a service via its gRPC port.

Thought for 15s

Given your constraints (Rust daemon, "all-inclusive" binary, ORT statically linked, Qdrant external), the Apple Silicon question reduces to: **can you realistically get acceleration beyond ORT CPU, without turning your packaging into a dylib bundle?**

What you get "for free" on Apple Silicon with your current approach

If you compile ONNX Runtime **natively for arm64**, you already benefit from Apple Silicon's fast CPU + NEON/Accelerate-class performance. For MiniLM-sized encoders, this is often "good enough" unless you're embedding at high throughput.

The main Apple-specific accelerator path in ONNX Runtime is CoreML EP

ONNX Runtime's **CoreML Execution Provider** is the supported way to target Apple hardware (CPU/GPU/ANE via Core ML). ONNX Runtime

Two practical gotchas for your "single static binary" packaging

1. **Provider libraries may still be dynamically loaded**
   ORT documentation notes it looks for provider shared libraries "in the same location as the executable" even when statically linked. ONNX Runtime
   So "static ORT" does not necessarily mean "no extra provider artifacts" once you enable non-CPU EPs (including CoreML).
2. **Static linkage + providers is a known pain point**
   There are longstanding issues where ORT tries to load `libonnxruntime_providers_shared` even when you're statically linking (reported for other EPs; the same architectural issue affects "single-

binary" expectations). GitHub

CoreML EP performance depends heavily on model shape constraints

CoreML EP can fall back to CPU if the model has unsupported/dynamic dimensions (batch/sequence). This has been explicitly called out (e.g., dynamic batch causing fallback). GitHub
Transformer encoders (MiniLM) are also a class where CoreML can be slower than CPU depending on operator coverage and graph fragmentation. GitHub

What I recommend for your daemon

Recommendation A (most practical): stay ORT CPU, optimize for Apple CPU

- Build **native arm64** and ensure no Rosetta anywhere.
- Focus on:
  - batching (small batches like 8–64 depending on latency)
  - tokenization cost (often dominates)
  - multithreading pinning / threadpool sizing
    This keeps your "single static binary" story intact and usually performs well on M-series for MiniLM-sized models.

Recommendation B (optional, Apple-optimized build): CoreML EP as a *separate* packaging target

Only if you can accept that "all-inclusive" may become "binary + adjacent provider dylibs" (or a more complex build).

If you do this:

- Build ORT with CoreML EP enabled (CoreML EP is the documented path). ONNX Runtime +1
- Ensure your exported ONNX has **static shapes** (at least sequence length; ideally batch too) to reduce CPU fallback risk. GitHub
- Add runtime selection:
  - Apple Silicon: try CoreML EP, fallback to CPU EP
  - Intel: CPU EP only

Recommendation C (bigger change, best Apple acceleration): Core ML / MLX backend

If you truly want to "benefit the most" from Apple accelerators, the cleanest route is usually **Core ML-native** or **MLX** inference, but that means maintaining a second model format/runtime outside ORT (and you lose the simplicity of "one ORT build everywhere"). (MLX is explicitly designed for Apple Silicon integration in the HF ecosystem.) Docs.rs +1

Concrete decision rule for your case

- If your embed throughput needs are moderate: **stick to ORT CPU** (native arm64) and invest in batching + tokenization + threadpool tuning.
- If you need much higher throughput *and* can tolerate a more complex macOS bundle: try **CoreML EP**, but only after you can export **static-shape ONNX** for the embedder and verify no fragmentation.

If you share your typical embedding workload (avg tokens/chunk, batch size, QPS target), I can tell you whether CoreML EP is likely to pay off or whether CPU tuning will get you 90% of the gain with 10% of the complexity.